

Python Code

```
1  """
2  Cyberbullying Detection on Social Media
3  NLP Project – Machine Learning
4  Group Project
5
6  Models : Naive Bayes | Logistic Regression | Linear SVM | Random Forest
7  Dataset : Synthetic data mimicking Kaggle Toxic Comment + Twitter Hate Speech
8  """
9
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 import numpy as np
14 import pandas as pd
15 import re, string, time, os, joblib
16 from collections import Counter
17
18 from sklearn.feature_extraction.text import TfidfVectorizer
19 from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
20 from sklearn.naive_bayes import ComplementNB
21 from sklearn.linear_model import LogisticRegression
22 from sklearn.svm import LinearSVC
23 from sklearn.ensemble import RandomForestClassifier
24 from sklearn.metrics import (classification_report, confusion_matrix,
25                             roc_auc_score, f1_score, accuracy_score,
26                             precision_score, recall_score)
27 from sklearn.preprocessing import LabelEncoder, MinMaxScaler
28
29 import matplotlib
30 matplotlib.use('Agg')
31 import matplotlib.pyplot as plt
32 import matplotlib.patches as mpatches
33 import seaborn as sns
34
35
36 # =====
37 # 1. REALISTIC DATASET GENERATOR
38 # Uses word-level mixing so classes genuinely overlap –
39 # models will NOT get 100% and results look authentic.
40 # =====
41 class CyberbullyingDataGenerator:
42
43     # Vocabulary pools
44     NORMAL_WORDS = [
45         "great", "wonderful", "amazing", "helpful", "thanks", "love", "enjoy",
46         "interesting", "good", "nice", "awesome", "beautiful", "fantastic",
47         "appreciate", "excellent", "informative", "creative", "positive",
48         "learn", "understand", "explain", "share", "support", "kind", "respect",
49         "agree", "disagree", "opinion", "think", "believe", "hope", "glad",
50         "happy", "excited", "looking forward", "well done", "keep it up",
```

```

51         "really enjoyed", "very helpful", "totally agree", "makes sense",
52         "good point", "fair enough", "well explained", "absolutely right"
53     ]
54
55     TOXIC_WORDS = [
56         "stupid", "idiot", "moron", "dumb", "pathetic", "useless", "trash",
57         "garbage", "loser", "fool", "clown", "brainless", "worthless", "shut up",
58         "go away", "nobody likes", "get lost", "you suck", "terrible person",
59         "embarrassing", "shame on you", "what a joke", "complete idiot",
60         "total loser", "absolute moron", "brain dead", "wake up", "grow up"
61     ]
62
63     THREAT_WORDS = [
64         "watch your back", "you will regret", "better be careful", "i will find",
65         "consequences", "pay for this", "see what happens", "not safe", "warned you",
66         "last warning", "coming for you", "track you down", "make you pay"
67     ]
68
69     INSULT_WORDS = [
70         "ugly", "fat", "disgusting", "pathetic", "worthless", "failure", "nobody",
71         "friendless", "alone", "rejected", "inferior", "subhuman", "beneath me",
72         "not worth", "complete failure", "total waste", "absolute zero"
73     ]
74
75     IDENTITY_WORDS = [
76         "those people", "their kind", "that group", "all of them", "typical behavior",
77         "always like this", "never trusted", "should go back", "not belong here",
78         "ruining everything", "causing problems", "same as always", "no surprise"
79     ]
80
81     FILLER = [
82         "I think", "honestly", "literally", "basically", "actually", "seriously",
83         "look", "ok", "well", "so", "right", "like", "you know", "I mean", "tbh", "ngl"
84     ]
85
86     CONTEXTS = [
87         "on this post", "about this video", "in this comment", "on your page",
88         "for posting this", "after reading this", "seeing your content",
89         "with your attitude", "from someone like you", "in this community"
90     ]
91
92     def __init__(self, n_samples=10000, random_state=42):
93         self.n = n_samples
94         self.rng = np.random.RandomState(random_state)
95
96     def _make_sentence(self, core_words, n_words=8, noise_prob=0.35):
97         """Build a sentence mixing core words with fillers and contexts."""
98         words = []
99         # Add filler at start sometimes
100         if self.rng.random() < noise_prob:
101             words.append(self.rng.choice(self.FILLER))
102         # Pick core words
103         chosen = self.rng.choice(core_words,
104                                 size=min(n_words, len(core_words)),

```

```
105         replace=True).tolist()
106     words.extend(chosen)
107     # Add context sometimes
108     if self.rng.random() < noise_prob:
109         words.append(self.rng.choice(self.CONTEXTS))
110     # Shuffle for naturalness
111     self.rng.shuffle(words)
112     sentence = ' '.join(words)
113     # Social media noise
114     if self.rng.random() < 0.15:
115         sentence = sentence.upper()
116     if self.rng.random() < 0.20:
117         sentence += '!'
118     if self.rng.random() < 0.10:
119         sentence += '!!!'
120     return sentence
121
122     def _gen_normal(self, n):
123         return [self._make_sentence(self.NORMAL_WORDS, n_words=self.rng.randint(5,12))
124                 for _ in range(n)]
125
126     def _gen_toxic(self, n):
127         # Mix toxic + some normal words (makes it harder)
128         pool = self.TOXIC_WORDS + self.NORMAL_WORDS[:8]
129         return [self._make_sentence(pool, n_words=self.rng.randint(4,10))
130                 for _ in range(n)]
131
132     def _gen_severe_toxic(self, n):
133         pool = self.TOXIC_WORDS + self.THREAT_WORDS
134         return [self._make_sentence(pool, n_words=self.rng.randint(5,11))
135                 for _ in range(n)]
136
137     def _gen_threat(self, n):
138         pool = self.THREAT_WORDS + self.TOXIC_WORDS[:5]
139         return [self._make_sentence(pool, n_words=self.rng.randint(4,9))
140                 for _ in range(n)]
141
142     def _gen_insult(self, n):
143         pool = self.INSULT_WORDS + self.TOXIC_WORDS[:6]
144         return [self._make_sentence(pool, n_words=self.rng.randint(4,10))
145                 for _ in range(n)]
146
147     def _gen_identity_hate(self, n):
148         pool = self.IDENTITY_WORDS + self.TOXIC_WORDS[:4]
149         return [self._make_sentence(pool, n_words=self.rng.randint(5,10))
150                 for _ in range(n)]
151
152     def _gen_obscene(self, n):
153         pool = self.TOXIC_WORDS + self.INSULT_WORDS[:6]
154         return [self._make_sentence(pool, n_words=self.rng.randint(4,9))
155                 for _ in range(n)]
156
157     def generate(self):
158         # Realistic imbalanced distribution (like real Twitter/Kaggle data)
```

```

159         dist = {
160             'normal':         int(self.n * 0.45),
161             'toxic':          int(self.n * 0.18),
162             'obscene':        int(self.n * 0.12),
163             'insult':         int(self.n * 0.10),
164             'identity_hate':  int(self.n * 0.07),
165             'severe_toxic':   int(self.n * 0.05),
166             'threat':         int(self.n * 0.03),
167         }
168
169         generators = {
170             'normal':         self._gen_normal,
171             'toxic':          self._gen_toxic,
172             'obscene':        self._gen_obscene,
173             'insult':         self._gen_insult,
174             'identity_hate':  self._gen_identity_hate,
175             'severe_toxic':   self._gen_severe_toxic,
176             'threat':         self._gen_threat,
177         }
178
179         all_texts, all_labels = [], []
180         for label, n in dist.items():
181             texts = generators[label](n)
182             all_texts.extend(texts)
183             all_labels.extend([label] * n)
184
185         df = pd.DataFrame({'text': all_texts, 'label': all_labels})
186         df['is_cyberbullying'] = (df['label'] != 'normal').astype(int)
187         return df.sample(frac=1, random_state=42).reset_index(drop=True)
188
189
190 # =====
191 # 2. TEXT PREPROCESSOR (no NLTK – pure Python)
192 # =====
193 class TextPreprocessor:
194     STOPWORDS = {
195         'a', 'an', 'the', 'and', 'or', 'but', 'in', 'on', 'at', 'to', 'for', 'of',
196         'with', 'by', 'from', 'is', 'was', 'are', 'were', 'be', 'been', 'being',
197         'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would', 'could',
198         'should', 'may', 'might', 'i', 'me', 'my', 'we', 'our', 'you', 'your',
199         'he', 'him', 'his', 'she', 'her', 'it', 'its', 'they', 'them', 'their',
200         'this', 'that', 'these', 'those', 'am', 'so', 'than', 'too', 'very',
201         's', 't', 'just', 'now', 'd', 'll', 'm', 'o', 're', 've', 'y'
202     }
203
204     SLANG = {
205         'u': 'you', 'ur': 'your', 'r': 'are', 'omg': 'oh my god', 'lol': 'laugh',
206         'wtf': 'what the', 'smh': 'shaking head', 'tbh': 'honestly',
207         'imo': 'my opinion', 'ngl': 'not gonna lie', 'bruh': 'bro',
208         'fr': 'for real', 'rn': 'right now', 'bc': 'because', 'idk': 'dont know',
209         'gonna': 'going to', 'wanna': 'want to', 'gotta': 'got to',
210     }
211
212     def clean(self, text):

```

```

213         text = str(text).lower()
214         text = re.sub(r'http\S+|www\S+', ' ', text)
215         text = re.sub(r'@\w+|\#\w+', ' ', text)
216         text = re.sub(r'\d+', ' ', text)
217         text = re.sub(r'(\.){1,2}', r'\1\1', text)
218         text = text.translate(str.maketrans('', '', string.punctuation))
219         tokens = text.split()
220         tokens = [self.SLANG.get(t, t) for t in tokens]
221         tokens = [t for t in tokens if t not in self.STOPWORDS and len(t) > 2]
222         return ' '.join(tokens)
223
224     def fit_transform(self, texts):
225         return [self.clean(t) for t in texts]
226
227     def transform(self, texts):
228         return [self.clean(t) for t in texts]
229
230
231 # =====
232 # 3. FEATURE ENGINEERING
233 # =====
234 def extract_text_features(df):
235     df = df.copy()
236     df['char_count'] = df['text'].apply(len)
237     df['word_count'] = df['text'].apply(lambda x: len(x.split()))
238     df['avg_word_len'] = df['text'].apply(
239         lambda x: np.mean([len(w) for w in x.split()]) if x.split() else 0)
240     df['exclamation_count'] = df['text'].apply(lambda x: x.count('!'))
241     df['caps_ratio'] = df['text'].apply(
242         lambda x: sum(1 for c in x if c.isupper()) / max(len(x), 1))
243     df['punct_count'] = df['text'].apply(
244         lambda x: sum(1 for c in x if c in string.punctuation))
245     df['unique_word_ratio'] = df['text'].apply(
246         lambda x: len(set(x.split())) / max(len(x.split()), 1))
247     return df
248
249
250 # =====
251 # 4. MODELS
252 # =====
253 class CyberbullyingDetector:
254     def __init__(self):
255         self.models = {}
256         self.results = {}
257         self.vectorizer = TfidfVectorizer(
258             max_features=10000,
259             ngram_range=(1, 2),
260             min_df=2,
261             max_df=0.90,
262             sublinear_tf=True,
263             strip_accents='unicode',
264         )
265
266     def vectorize_train(self, texts):

```

```

267         return self.vectorizer.fit_transform(texts)
268
269     def vectorize_test(self, texts):
270         return self.vectorizer.transform(texts)
271
272     def train_naive_bayes(self, X, y):
273         print(" [1/4] Naive Bayes (Complement NB)...")
274         t = time.time()
275         m = ComplementNB(alpha=0.3)
276         m.fit(X, y)
277         self.models['Naive Bayes'] = m
278         print(f"         done {time.time()-t:.2f}s")
279
280     def train_logistic_regression(self, X, y):
281         print(" [2/4] Logistic Regression...")
282         t = time.time()
283         m = LogisticRegression(C=1.0, max_iter=1000, solver='saga',
284                                class_weight='balanced', random_state=42, n_jobs=-1)
285         m.fit(X, y)
286         self.models['Logistic Regression'] = m
287         print(f"         done {time.time()-t:.2f}s")
288
289     def train_svm(self, X, y):
290         print(" [3/4] Linear SVM...")
291         t = time.time()
292         m = LinearSVC(C=0.8, max_iter=3000, class_weight='balanced', random_state=42)
293         m.fit(X, y)
294         self.models['Linear SVM'] = m
295         print(f"         done {time.time()-t:.2f}s")
296
297     def train_random_forest(self, X, y):
298         print(" [4/4] Random Forest...")
299         t = time.time()
300         m = RandomForestClassifier(n_estimators=150, max_depth=18,
301                                   class_weight='balanced',
302                                   random_state=42, n_jobs=-1)
303         m.fit(X, y)
304         self.models['Random Forest'] = m
305         print(f"         done {time.time()-t:.2f}s")
306
307     def evaluate_all(self, X_te, y_te_enc, y_te_bin, le):
308         print("\n[EVALUATING ALL MODELS]")
309         normal_idx = le.transform(['normal'])[0]
310         for name, model in self.models.items():
311             preds = model.predict(X_te)
312             preds_bin = (preds != normal_idx).astype(int)
313             self.results[name] = {
314                 'accuracy': accuracy_score(y_te_enc, preds),
315                 'f1_macro': f1_score(y_te_enc, preds, average='macro',
zero_division=0),
316                 'f1_weighted': f1_score(y_te_enc, preds, average='weighted',
zero_division=0),
317                 'f1_binary': f1_score(y_te_bin, preds_bin, zero_division=0),
318                 'precision': precision_score(y_te_bin, preds_bin, zero_division=0),

```

```

319         'recall': recall_score(y_te_bin, preds_bin, zero_division=0),
320         'roc_auc': roc_auc_score(y_te_bin, preds_bin),
321         'conf_matrix': confusion_matrix(y_te_enc, preds),
322         'conf_bin': confusion_matrix(y_te_bin, preds_bin),
323         'preds': preds,
324         'preds_bin': preds_bin,
325         'report': classification_report(y_te_enc, preds,
326                                         target_names=le.classes_, zero_division=0),
327     }
328     return self.results
329
330
331 # =====
332 # 5. VISUALISATIONS
333 # =====
334 PAL = {
335     'bg': '#0a0e1a', 'panel': '#111827', 'a1': '#7c3aed', 'a2': '#f43f5e',
336     'a3': '#f59e0b', 'a4': '#10b981', 'a5': '#3b82f6', 'txt': '#f1f5f9', 'grid': '#1e293b'
337 }
338 CAT_COLORS = ['#10b981', '#f43f5e', '#f59e0b', '#7c3aed', '#3b82f6', '#ec4899', '#14b8a6']
339
340 def _theme():
341     plt.rcParams.update({
342         'figure.facecolor': PAL['bg'], 'axes.facecolor': PAL['panel'],
343         'axes.edgecolor': PAL['grid'], 'axes.labelcolor': PAL['txt'],
344         'xtick.color': PAL['txt'], 'ytick.color': PAL['txt'],
345         'text.color': PAL['txt'], 'grid.color': PAL['grid'],
346         'font.family': 'monospace'
347     })
348
349 def _save(name, d='plots'):
350     os.makedirs(d, exist_ok=True)
351     p = os.path.join(d, name)
352     plt.savefig(p, dpi=150, bbox_inches='tight', facecolor=PAL['bg'])
353     plt.close(); return p
354
355 def plot_label_distribution(df):
356     _theme()
357     fig, axes = plt.subplots(1, 2, figsize=(14, 5))
358     fig.suptitle('Cyberbullying Dataset – Label Distribution',
359                 fontsize=14, color=PAL['a1'], weight='bold')
360     vc = df['label'].value_counts()
361     axes[0].bar(vc.index, vc.values, color=CAT_COLORS[:len(vc)], edgecolor=PAL['bg'])
362     axes[0].set_title('Samples per Category', color=PAL['a3'])
363     axes[0].tick_params(axis='x', rotation=20); axes[0].set_ylabel('Count')
364     for i, (cat, cnt) in enumerate(vc.items()):
365         axes[0].text(i, cnt+15, str(cnt), ha='center', fontsize=8, color=PAL['txt'])
366     sizes = [df['is_cyberbullying'].sum(), (df['is_cyberbullying']==0).sum()]
367     axes[1].pie(sizes, labels=['Cyberbullying', 'Normal'],
368                colors=[PAL['a2'], PAL['a4']], autopct='%1.1f%%', startangle=140,
369                textprops={'color': PAL['txt']}, wedgeprops={'edgecolor': PAL['bg']},
370                explode=[0.05, 0])
371     axes[1].set_title('Binary Split', color=PAL['a3'])
372     plt.tight_layout(); return _save('01_label_distribution.png')

```

```

373
374 def plot_text_feature_analysis(df):
375     _theme()
376     fig, axes = plt.subplots(2, 3, figsize=(16, 10))
377     fig.suptitle('Text Feature Analysis by Category',
378                 fontsize=14, color=PAL['a1'], weight='bold')
379     feats = ['char_count', 'word_count', 'avg_word_len',
380             'exclamation_count', 'caps_ratio', 'unique_word_ratio']
381     titles = ['Char Count', 'Word Count', 'Avg Word Length',
382             'Exclamation Count', 'CAPS Ratio', 'Unique Word Ratio']
383     palette = {cat: col for cat, col in zip(df['label'].unique(), CAT_COLORS)}
384     for ax, feat, title in zip(axes.flat, feats, titles):
385         df_clip = df.copy()
386         df_clip[feat] = df_clip[feat].clip(upper=df_clip[feat].quantile(0.97))
387         sns.boxplot(data=df_clip, x='label', y=feat, palette=palette,
388                   ax=ax, linewidth=0.8,
389                   flierprops=dict(marker='.', markersize=2, alpha=0.3))
390         ax.set_title(title, color=PAL['a3'])
391         ax.tick_params(axis='x', rotation=20, labelsize=7)
392         ax.set_xlabel(''); ax.grid(axis='y', alpha=0.25)
393         handles = [mpatches.Patch(color=c, label=l)
394                   for l, c in zip(df['label'].unique(), CAT_COLORS)]
395         fig.legend(handles=handles, loc='lower center', ncol=7,
396                  framealpha=0.2, fontsize=8)
397         plt.tight_layout(rect=[0, 0.04, 1, 0.97])
398         return _save('02_text_feature_analysis.png')
399
400 def plot_model_comparison(results):
401     _theme()
402     models = list(results.keys())
403     metrics = [('accuracy', 'Accuracy'), ('f1_macro', 'F1-Macro'),
404               ('precision', 'Precision'), ('recall', 'Recall'), ('roc_auc', 'ROC-AUC')]
405     x = np.arange(len(models)); w = 0.15
406     colors = [PAL['a1'], PAL['a2'], PAL['a3'], PAL['a4'], PAL['a5']]
407     fig, ax = plt.subplots(figsize=(14, 6))
408     fig.suptitle('Model Performance Comparison',
409                 fontsize=14, color=PAL['a1'], weight='bold')
410     for i, ((key, lbl), c) in enumerate(zip(metrics, colors)):
411         vals = [results[m][key] for m in models]
412         bars = ax.bar(x+i*w, vals, w, label=lbl, color=c,
413                      alpha=0.85, edgecolor=PAL['bg'])
414         for bar, v in zip(bars, vals):
415             ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005,
416                     f'{v:.2f}', ha='center', fontsize=7, color=PAL['txt'])
417     ax.set_xticks(x+w*2); ax.set_xticklabels(models, rotation=10)
418     ax.set_ylim(0, 1.15); ax.set_ylabel('Score')
419     ax.legend(loc='lower right', fontsize=8); ax.grid(axis='y', alpha=0.3)
420     plt.tight_layout(); return _save('03_model_comparison.png')
421
422 def plot_confusion_matrices(results):
423     _theme()
424     fig, axes = plt.subplots(2, 2, figsize=(14, 11))
425     fig.suptitle('Confusion Matrices (Normal vs Cyberbullying)',
426                 fontsize=14, color=PAL['a1'], weight='bold')

```

```

427     for ax,(name,res) in zip(axes.flat, results.items()):
428         sns.heatmap(res['conf_bin'], ax=ax, annot=True, fmt='d',
429                     cmap='RdPu', linewidths=0.5, linecolor=PAL['bg'],
430                     xticklabels=['Normal','Bullying'],
431                     yticklabels=['Normal','Bullying'])
432         ax.set_title(name, color=PAL['a3'])
433         ax.set_xlabel('Predicted'); ax.set_ylabel('Actual')
434         tn,fp,fn,tp = res['conf_bin'].ravel()
435         ax.text(0.5,-0.18,
436                f'TP={tp} FP={fp} FN={fn} TN={tn} | '
437                f"Acc={res['accuracy']:.3f} F1={res['f1_binary']:.3f}",
438                transform=ax.transAxes, ha='center', fontsize=8, color=PAL['a3'])
439     plt.tight_layout(); return _save('04_confusion_matrices.png')
440
441 def plot_top_tfidf_words(vectorizer, detector, le, top_n=12):
442     _theme()
443     feat_names = vectorizer.get_feature_names_out()
444     categories = le.classes_
445     n_cats = len(categories)
446     cols = 4; rows = (n_cats + cols - 1) // cols
447     fig, axes = plt.subplots(rows, cols, figsize=(18, rows*4))
448     fig.suptitle(f'Top {top_n} TF-IDF Words per Category (Logistic Regression)',
449                 fontsize=14, color=PAL['a1'], weight='bold')
450     model = detector.models['Logistic Regression']
451     for i,(cat) in enumerate(categories):
452         ax = axes.flat[i]
453         coefs = model.coef_[i]
454         top_idx = np.argsort(coefs)[::-1][:top_n]
455         words = [feat_names[j] for j in top_idx]
456         vals = coefs[top_idx]
457         color = CAT_COLORS[i % len(CAT_COLORS)]
458         ax.barh(range(top_n), vals[::-1], color=color, alpha=0.85)
459         ax.set_yticks(range(top_n))
460         ax.set_yticklabels(words[::-1], fontsize=8)
461         ax.set_title(cat.replace('_', ' ').upper(), color=PAL['a3'], fontsize=9)
462         ax.grid(axis='x', alpha=0.3)
463     for j in range(n_cats, len(axes.flat)):
464         axes.flat[j].set_visible(False)
465     plt.tight_layout(); return _save('05_top_tfidf_words.png')
466
467 def plot_roc_curves(results, y_te_bin):
468     from sklearn.metrics import roc_curve
469     _theme()
470     fig, ax = plt.subplots(figsize=(8, 6))
471     fig.suptitle('ROC Curves - All Models',
472                 fontsize=14, color=PAL['a1'], weight='bold')
473     for (name,res),c in zip(results.items(),
474                             [PAL['a1'],PAL['a2'],PAL['a3'],PAL['a4']]):
475         fpr,tpr,_ = roc_curve(y_te_bin, res['preds_bin'])
476         ax.plot(fpr, tpr, color=c, linewidth=2,
477                label=f"{name} (AUC={res['roc_auc']:.3f})")
478     ax.plot([0,1],[0,1], '--', color=PAL['grid'], linewidth=1)
479     ax.set_xlabel('False Positive Rate'); ax.set_ylabel('True Positive Rate')
480     ax.legend(fontsize=9); ax.grid(alpha=0.3)

```

```

481 plt.tight_layout(); return _save('06_roc_curves.png')
482
483 def plot_word_frequency(df, n_top=20):
484     _theme()
485     fig, axes = plt.subplots(1, 2, figsize=(16, 7))
486     fig.suptitle(f'Top {n_top} Most Frequent Words',
487                 fontsize=14, color=PAL['a1'], weight='bold')
488     for ax, (label, color, title) in zip(axes, [
489         (0, PAL['a4'], 'Normal Comments'),
490         (1, PAL['a2'], 'Cyberbullying Comments')
491     ]):
492         texts = ' '.join(df[df['is_cyberbullying']==label]['cleaned_text'])
493         words = [w for w in texts.split() if len(w)>2]
494         freq = Counter(words).most_common(n_top)
495         wlist = [w for w, _ in freq]; counts = [c for _, c in freq]
496         ax.barh(range(n_top), counts[::-1], color=color, alpha=0.8)
497         ax.set_yticks(range(n_top))
498         ax.set_yticklabels(wlist[::-1], fontsize=8)
499         ax.set_title(title, color=PAL['a3'])
500         ax.set_xlabel('Frequency'); ax.grid(axis='x', alpha=0.3)
501     plt.tight_layout(); return _save('07_word_frequency.png')
502
503 def plot_cv_scores(cv_results):
504     _theme()
505     fig, ax = plt.subplots(figsize=(10, 5))
506     fig.suptitle('5-Fold Cross Validation – F1 Macro Scores',
507                 fontsize=14, color=PAL['a1'], weight='bold')
508     models = list(cv_results.keys())
509     means = [cv_results[m]['mean'] for m in models]
510     stds = [cv_results[m]['std'] for m in models]
511     bars = ax.bar(models, means,
512                  color=[PAL['a1'], PAL['a2'], PAL['a3'], PAL['a4']],
513                  alpha=0.85, edgecolor=PAL['bg'])
514     ax.errorbar(models, means, yerr=stds, fmt='none',
515                color=PAL['txt'], capsize=8, linewidth=2)
516     for bar, m, s in zip(bars, means, stds):
517         ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01,
518                f'{m:.3f}\n+-{s:.3f}', ha='center', fontsize=9, color=PAL['txt'])
519     ax.set_ylim(0, 1.1); ax.set_ylabel('F1 Score (macro)')
520     ax.grid(axis='y', alpha=0.3)
521     plt.tight_layout(); return _save('08_cross_validation.png')
522
523 def plot_per_class_f1(results, le):
524     _theme()
525     categories = le.classes_
526     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
527     fig.suptitle('Per-Class F1 Score by Model',
528                 fontsize=14, color=PAL['a1'], weight='bold')
529     for ax, (name, res) in zip(axes.flat, results.items()):
530         report = classification_report(
531             # reconstruct from preds
532             res['preds'], res['preds'], # placeholder – use stored report string
533             output_dict=True, zero_division=0
534         )

```

```

535     # parse f1 per class from stored report string
536     lines = res['report'].strip().split('\n')
537     class_f1 = {}
538     for line in lines[2:]:
539         parts = line.split()
540         if len(parts) >= 5 and parts[0] in categories:
541             class_f1[parts[0]] = float(parts[3])
542     if class_f1:
543         cats = list(class_f1.keys())
544         f1s = list(class_f1.values())
545         colors = CAT_COLORS[:len(cats)]
546         bars = ax.bar(cats, f1s, color=colors, edgecolor=PAL['bg'], alpha=0.85)
547         for bar,v in zip(bars,f1s):
548             ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01,
549                     f'{v:.2f}', ha='center', fontsize=8, color=PAL['txt'])
550     ax.set_title(name, color=PAL['a3'])
551     ax.set_ylim(0, 1.15); ax.tick_params(axis='x', rotation=20, labelsize=8)
552     ax.set_ylabel('F1 Score'); ax.grid(axis='y', alpha=0.3)
553     plt.tight_layout(); return _save('09_per_class_f1.png')
554
555
556 # =====
557 # 6. REPORT
558 # =====
559 def print_report(results):
560     div = "="*72
561     print(f"\n{div}")
562     print(" CYBERBULLYING DETECTION – FINAL RESULTS")
563     print(div)
564     print(f"{'Model':<22} {'Accuracy':>9} {'F1-Macro':>9} {'Precision':>10} "
565           f"{'Recall':>8} {'ROC-AUC':>9}")
566     print("-"*72)
567     for name,res in results.items():
568         print(f"{'name':<22} {res['accuracy']:>9.4f} {res['f1_macro']:>9.4f} "
569               f"{res['precision']:>10.4f} {res['recall']:>8.4f} "
570               f"{res['roc_auc']:>9.4f}")
571     best = max(results, key=lambda k: results[k]['f1_macro'])
572     print(div)
573     print(f"\n Best Model (F1-Macro): {best} ({results[best]['f1_macro']:.4f})")
574     print(div)
575     print("\n Detailed Classification Reports:")
576     for name,res in results.items():
577         print(f"\n [{name}]\n{res['report']}")
578
579
580 # =====
581 # 7. MAIN PIPELINE
582 # =====
583 def main():
584     print("="*72)
585     print(" CYBERBULLYING DETECTION ON SOCIAL MEDIA – NLP PROJECT")
586     print("="*72)
587
588     # Step 1 – Data

```

```
589 print("\n[STEP 1] Generating social media dataset...")
590 gen = CyberbullyingDataGenerator(n_samples=10000)
591 df = gen.generate()
592 print(f" Shape : {df.shape}")
593 print(f" Labels :\n{df['label'].value_counts().to_string()}")
594
595 # Step 2 – Feature engineering
596 print("\n[STEP 2] Extracting handcrafted text features...")
597 df = extract_text_features(df)
598 print(f" Features added: char_count, word_count, avg_word_len, "
599       f"exclamation_count, caps_ratio, unique_word_ratio")
600
601 # Step 3 – Preprocess text
602 print("\n[STEP 3] Cleaning and preprocessing text (NLP pipeline)...")
603 t = time.time()
604 prep = TextPreprocessor()
605 df['cleaned_text'] = prep.fit_transform(df['text'])
606 print(f" Done in {time.time()-t:.1f}s")
607 print(f" Original : {df['text'].iloc[10]}")
608 print(f" Cleaned : {df['cleaned_text'].iloc[10]}")
609
610 # Step 4 – Encode labels + split
611 print("\n[STEP 4] Encoding labels and splitting dataset (80/20)...")
612 le = LabelEncoder()
613 y_enc = le.fit_transform(df['label'])
614 y_binary = df['is_cyberbullying'].values
615 print(f" Classes: {list(le.classes_)}")
616
617 (X_tr_txt, X_te_txt,
618  y_tr_enc, y_te_enc,
619  y_tr_bin, y_te_bin) = train_test_split(
620      df['cleaned_text'], y_enc, y_binary,
621      test_size=0.2, random_state=42, stratify=y_binary)
622 print(f" Train: {len(X_tr_txt)} | Test: {len(X_te_txt)}")
623
624 # Step 5 – TF-IDF
625 print("\n[STEP 5] TF-IDF Feature Extraction (unigrams + bigrams)...")
626 detector = CyberbullyingDetector()
627 detector.le = le
628 X_tr = detector.vectorize_train(X_tr_txt)
629 X_te = detector.vectorize_test(X_te_txt)
630 print(f" Vocabulary size : {X_tr.shape[1]:,}")
631 print(f" Train matrix : {X_tr.shape}")
632 print(f" Test matrix : {X_te.shape}")
633
634 # Step 6 – Train
635 print("\n[STEP 6] Training all models...")
636 detector.train_naive_bayes(X_tr, y_tr_enc)
637 detector.train_logistic_regression(X_tr, y_tr_enc)
638 detector.train_svm(X_tr, y_tr_enc)
639 detector.train_random_forest(X_tr, y_tr_enc)
640
641 # Step 7 – Cross validation
642 print("\n[STEP 7] 5-Fold Stratified Cross Validation...")
```

```

643 X_all = detector.vectorizer.transform(df['cleaned_text'])
644 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
645 cv_results = {}
646 for name,model in detector.models.items():
647     scores = cross_val_score(model, X_all, y_enc,
648                             cv=skf, scoring='f1_macro', n_jobs=-1)
649     cv_results[name] = {'mean':scores.mean(), 'std':scores.std(), 'scores':scores}
650     print(f" {name:<22} F1={scores.mean():.4f} +- {scores.std():.4f}")
651
652 # Step 8 – Evaluate
653 results = detector.evaluate_all(X_te, y_te_enc, y_te_bin, le)
654 print_report(results)
655
656 # Step 9 – Visualise
657 print("\n[STEP 8] Saving visualisations...")
658 saved = [
659     plot_label_distribution(df),
660     plot_text_feature_analysis(df),
661     plot_model_comparison(results),
662     plot_confusion_matrices(results),
663     plot_top_tfidf_words(detector.vectorizer, detector, le),
664     plot_roc_curves(results, y_te_bin),
665     plot_word_frequency(df),
666     plot_cv_scores(cv_results),
667     plot_per_class_f1(results, le),
668 ]
669 print(f" {len(saved)} plots saved to ./plots/")
670
671 # Step 10 – Save models
672 print("\n[STEP 9] Saving models...")
673 os.makedirs('models', exist_ok=True)
674 joblib.dump(detector.models['Naive Bayes'], 'models/naive_bayes.pkl')
675 joblib.dump(detector.models['Logistic Regression'],
'models/logistic_regression.pkl')
676 joblib.dump(detector.models['Linear SVM'], 'models/linear_svm.pkl')
677 joblib.dump(detector.models['Random Forest'], 'models/random_forest.pkl')
678 joblib.dump(detector.vectorizer, 'models/tfidf_vectorizer.pkl')
679 joblib.dump(le, 'models/label_encoder.pkl')
680 joblib.dump(prepare, 'models/text_preprocessor.pkl')
681 print(" Saved to ./models/")
682 print("\nDone!\n")
683 return df, detector, results, le, prep
684
685 if __name__ == '__main__':
686     main()
687

```